

Infografía

Módulo 7 — Godot 4 · Sesión 5.6

IA de navegación (2): pathfinding con tilemaps

UPB · Gestión I/2026

Plan de la clase

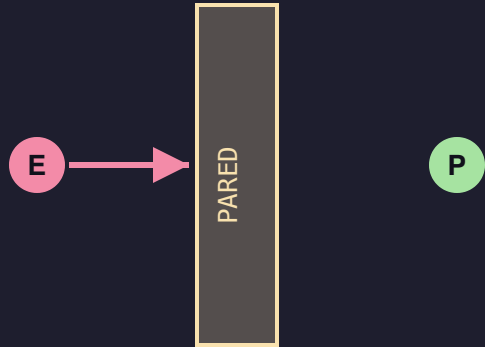
1. **El problema** — steering choca; necesitamos el camino.
2. **La grilla** — `AStarGrid2D` desde el tilemap (escena 03).
3. **Seguir el camino** — A^* y `get_id_path` (escena 04).
4. **Recalcular bien** — no 60 veces por segundo (escena 05).
5. **Terreno con costo** — pesos por celda (ejercicio 06).
6. **El camino del motor** — `NavigationAgent2D` (escena 07).

*El mundo de tiles de la sesión 5 ahora es la **grilla por la que se navega**.*

El problema: dirección ≠ camino

Steering (reactivo)

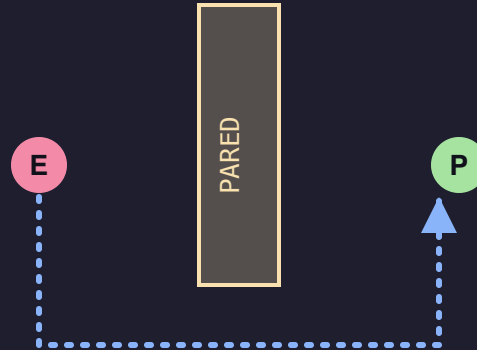
conoce la DIRECCIÓN, no el camino



✱ se traba

Pathfinding (A*)

calcula el CAMINO y rodea



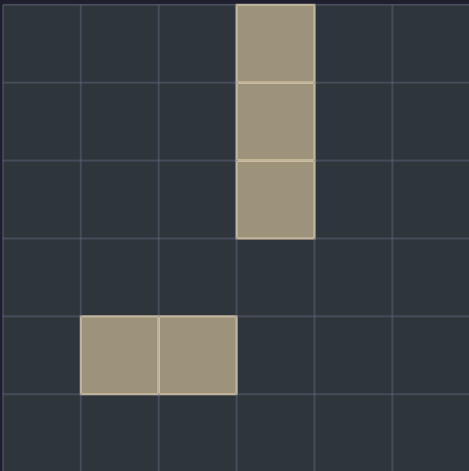
✓ llega rodeando

mismo mundo · misma meta · dos formas de moverse

El enemigo de la sesión 5.5 conoce hacia dónde está el jugador, pero no cómo llegar. Hoy: que calcule el camino.

La idea: el tilemap ES el dato de navegación

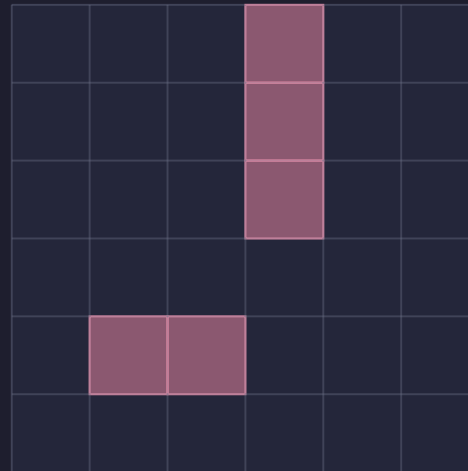
TileMapLayer "Paredes"



`get_used_cells()`

`set_point_solid()`

AStarGrid2D



la pared pintada SE VUELVE el obstáculo del A* – no se duplica nada

Parte 1 — construir la grilla (escena 03)

`AStarGrid2D` es una grilla que sabe qué celdas están bloqueadas.

```
var astar := AStarGrid2D.new()
astar.region = Rect2i(Vector2i.ZERO, tam_del_mapa)
astar.cell_size = Vector2(16, 16)
astar.update()

for celda in paredes.get_used_cells(): # ¡las paredes que ya pintaste!
    astar.set_point_solid(celda, true)
```

Sin duplicar nada: el mismo TileMapLayer de paredes alimenta el A.*

Parte 2 — pedir y seguir el camino (escena 04)

```
var ini := paredes.local_to_map(paredes.to_local(global_position))
var fin := paredes.local_to_map(paredes.to_local(player.global_position))
var camino := astar.get_id_path(ini, fin) # lista de celdas
```

Seguir el camino = apuntar al siguiente punto; al pisarlo, avanzar al que sigue.

```
var objetivo := map_to_local(camino[idx])
if global_position.distance_to(objetivo) < 3.0:
    idx += 1
velocity = global_position.direction_to(objetivo) * max_speed
```

Ahora el enemigo rodea el muro. Reusamos el "ir hacia" de la 5.5.

Parte 2 — píxeles ↔ celdas

Dos funciones del TileMapLayer hacen toda la traducción:

```
local_to_map(pos)
```

píxeles → **celda** (col, fila)

```
map_to_local(celda)
```

celda → **centro** en píxeles

A* piensa en **celdas**.

El mundo está en **píxeles**.

Entre los dos, siempre
estas dos funciones.

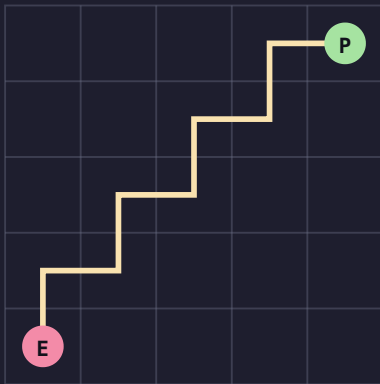
Son las mismas de la sesión 5 ("leer el mapa"). Acá, para navegar.

Parte 3 – diagonales

`diagonal_mode` decide si se puede cortar en diagonal.

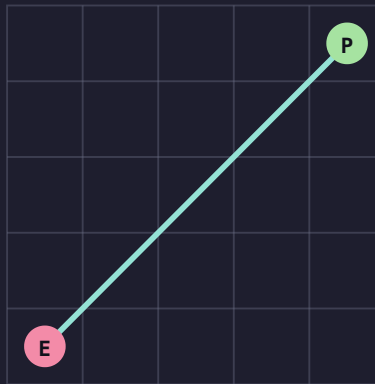
`diagonal_mode`: ¿se puede ir en diagonal?

`DIAGONAL_MODE_NEVER`



4 direcciones · escalonado

`...ONLY_IF_NO_OBSTACLES`



8 direcciones · más directo

`DIAGONAL_MODE_NEVER` (4 dir, escalonado) vs `...ONLY_IF_NO_OBSTACLES` (8 dir, directo).

Parte 4 — recalcular, pero bien (escena 05)

El jugador se mueve → hay que recalcular. Pero un A* es **caro**: hacerlo 60 veces por segundo, por cada enemigo, no escala.

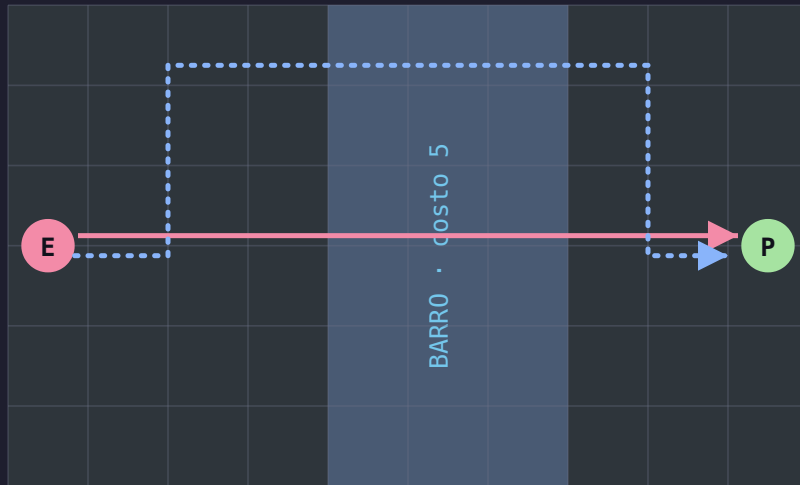
```
_acum += delta
var celda_jugador := _cel(player.global_position)
if _acum >= recalcular_cada or celda_jugador != _ultima_celda:
    _acum = 0.0
    _ultima_celda = celda_jugador
    _recalcular()
```

🔧 Lo completamos en vivo: recalcular cada tanto, o al cambiar de celda.

Parte 5 — no todo el suelo es igual (ejercicio 06)

Cruzar barro debería costar más que ir por el pasto. `set_point_weight_scale` le pone un **peso** a cada celda: A* busca el más **barato**, no el más corto.

A* busca el camino más BARATO, no el más corto



— sin pesos: cruza el barro (corto)

... con pesos: lo rodea (barato)

Parte 5 — el costo sale del tile

Es "leer el mapa" de la sesión 5, ahora para navegar: cada tile de suelo lleva un dato `costo` (pasto = 1, barro = 5).

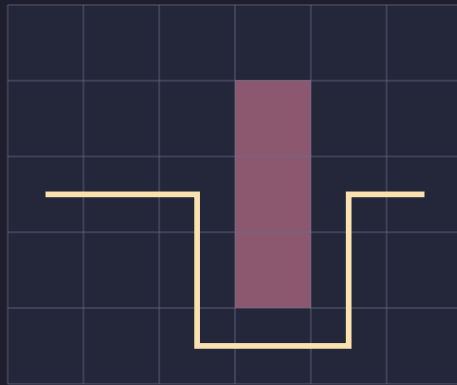
```
for celda in todas_las_celdas:
    var datos := suelo.get_cell_tile_data(celda)    # ¡puede ser null!
    if datos:
        astar.set_point_weight_scale(celda, datos.get_custom_data("costo"))
```

🎓 Tu turno: completá los 3 `TODO` de `enemigo_terreno.gd`.

Parte 6 — el camino del motor (escena 07)

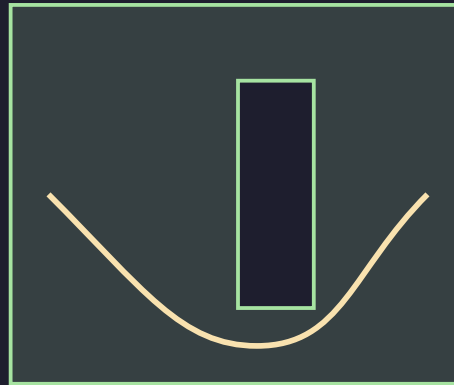
Dos formas de representar "por dónde se puede caminar"

Grilla — AStarGrid2D



celdas · transparente, lo ves
camino "escalonado"

Navmesh — NavigationAgent2D



polígono · suave + evasión gratis
camino "natural", pero más opaco

Parte 6 – NavigationAgent2D

Mismo trabajo, a la manera de Godot: le das un destino y te da el siguiente paso.

```
agente.target_position = player.global_position
if not agente.is_navigation_finished():
    var paso := agente.get_next_path_position()
    velocity = global_position.direction_to(paso) * max_speed
```

Camino suave + evasión entre agentes gratis. A cambio, el navmesh se ve menos que una grilla.

Resumen

El **tilemap** es la grilla de navegación.

`AStarGrid2D`: celdas sólidas (paredes) + pesos (terreno).

`get_id_path` da el camino; lo seguís celda a celda.

Recalculás **cada tanto**, no cada frame.

Grilla transparente (A^) o navmesh del motor (`NavigationAgent2D`): dos herramientas, mismo objetivo.*